

Promo Code Module — Mobile App Testing Checklist

Cross-platform (iOS + Android) native app — extends the completed web/admin test pass

Scope: Native cross-platform app (React Native/Flutter-style), backend shared with jt-frontend.scopethinkers.com

Builds on: Promo Code Module — QA Report (web), 21 July 2026

Purpose: Cover mobile-specific behaviors not exercised by browser-based testing — not a re-run of every web case

This checklist assumes the core redemption logic (discount math, expiry/usage-limit enforcement, case-insensitivity) was already validated on web and should behave identically server-side. The items below focus on what changes purely because the client is a native mobile app: input handling, app lifecycle, platform divergence, and native payment integration.

0. Pre-Test Setup

- ☐ Confirm which promo codes from the web test pass are still live (QATEST_PERCENT10, QATEST_FLAT100, etc.) — reuse them for parity checks instead of recreating
- ☐ Confirm app build/version number and whether it points at the same backend as jt-frontend.scopethinkers.com
- ☐ Set up test devices: at least 1 iOS + 1 Android, including one low-end/older OS version each
- ☐ Confirm test account (or fresh qatest_promo_mobile_[date]@mailinator.com) can log in on the app

1. Functional Parity

Re-run the web module's core redemption cases inside the app to confirm identical server-side behavior.

- M1** ☐ Apply a percentage code (QATEST_PERCENT10) — discount math correct, success message shown
- M2** ☐ Apply a flat-amount code (QATEST_FLAT100) — total correct
- M3** ☐ Apply single-use code, then attempt second use — rejected
- M4** ☐ Apply invalid/nonexistent code — clean error, no crash
- M5** ☐ Apply expired/disabled code — rejected with clear message
- M6** ☐ Remove an applied code — original price restored
- M7** ☐ Case sensitivity (lowercase vs uppercase) — matches web behavior

- M8** ☐ 100% discount code — checkout goes to ₹0; confirm whether the same "Amount Paid" display bug (JP3P-111) reproduces in-app
-
- M9** ☐ Admin-side usage count increments after an app-side redemption (cross-check web admin panel)

2. Native Input & Keyboard Behavior

- M10** ☐ Promo code field opens the correct keyboard type (alphanumeric, not numeric-only)
-
- M11** ☐ Autocapitalize/autocorrect doesn't mangle the code as typed (no inserted space or "corrected" word)
-
- M12** ☐ Paste a copied code from clipboard (e.g. from an SMS/email promo) works correctly
-
- M13** ☐ Voice-to-text dictation into the promo field is handled or at least doesn't crash
-
- M14** ☐ Field-clearing "X" button (if present) works and resets validation state

3. App Lifecycle & Interruptions

- M15** ☐ Apply a promo code, background the app, return — code and discounted total persist correctly
-
- M16** ☐ Apply a code, receive an incoming call/notification overlay, dismiss it, return — state intact
-
- M17** ☐ Force-kill the app mid-checkout (after applying code, before paying) and relaunch — no stale discounted total shown without the code actually still applied server-side
-
- M18** ☐ Rotate device (if orientation unlocked) while checkout is open — layout and applied state survive
-
- M19** ☐ Lose network connectivity right after tapping "Apply" — clear error/timeout, not an infinite spinner
-
- M20** ☐ Lose network connectivity mid-payment (after redirecting to gateway) — no double-charge, no stuck order state on reconnect

4. Platform-Specific Divergence (iOS vs Android)

- M21** ☐ Android hardware/gesture "Back" during checkout with a code applied — clean exit, no broken state
-
- M22** ☐ iOS swipe-back gesture during checkout — same check
-
- M23** ☐ Compare ₹ currency symbol and discount amount rendering across both platforms — no clipping or fallback glyphs
-
- M24** ☐ Dark mode on both platforms — success/error banners remain legible (contrast)
-
- M25** ☐ Large accessibility text scaling — promo UI doesn't break or truncate the discount summary

5. Payment Gateway on Mobile

- M26** ☐ Confirm whether the app uses Razorpay's native SDK or a webview — behavior may differ from the already-tested browser flow
- M27** ☐ UPI intent flow (Android): "Pay" launches an installed UPI app (GPay/PhonePe); completing there and returning resolves the order correctly
- M28** ☐ Cancelling from the UPI/bank app and returning — no false success shown, promo code remains applied for retry
- M29** ☐ Zero-amount (100%-discount) checkout on mobile — confirm gateway/SDK is skipped entirely, same as web

6. Push Notifications / Confirmations

- M30** ☐ Purchase-confirmation push notification (if any) states the correct discounted amount, not the original price — mobile analogue of JP3P-111
- M31** ☐ Tapping the push notification deep-links to the correct order/course, not a generic screen

7. Regression Checks Specific to App Rendering

- M32** ☐ Long promo code names (near the 50-char DB limit) don't overflow/truncate awkwardly in the "applied code" chip/label
- M33** ☐ Screen reader (VoiceOver / TalkBack) reads the promo field, applied-discount confirmation, and error messages correctly
- M34** ☐ Small-screen devices (SE-sized iPhone, budget Android) — promo field and Apply button aren't hidden behind the keyboard or cut off

8. Cross-Module Consistency (Mobile ↔ Admin ↔ Web)

- M35** ☐ A code redeemed on the app shows up correctly in the same admin GST Report as web-redeemed purchases (same reconciliation gap as JP3P-112 likely applies)
- M36** ☐ A code disabled/edited on the web admin panel while the app has a stale cached promo list — reapplying in-app correctly picks up the new state